DuocUC | ONLINE



Guía de aprendizaje

Análisis y aplicación del patrón arquitectónico con Backend for Frontend (BFF)

Exp 2 – Semana 4

Desarrollo Backend III (PBY2203)

Facilitador disciplinar: Ana Karina Villagrán Ibarra

Asesor par: Ignacio Villarroel

Índice

Introducción a la semana....3
Resultado de aprendizaje....4
Conceptos relevantes....4
Preguntas activadoras....5
Actividad....5
Analizando el patrón Backend for Frontend (BFF)....6
Introducción....6
Contexto de origen....8
¿Qué es el patrón BFF?....9
¿Para qué se usa?....10
¿Cómo se utiliza?....12
Impacto de no tener un BFF....12
Caso ejemplo de monolito....14
Estrategias de implementación de BFF....15
Cierre de la semana....17
Referencias....18
Lecturas de la semana....18
Apuntes....19

Introducción a la semana



Esta semana, te centrarás en comprender y aplicar el patrón arquitectónico Backend for Frontend (BFF). Este patrón es ideal para optimizar la interacción entre diferentes frontends (por ejemplo, aplicaciones web y móviles) y backends, permitiéndote desarrollar soluciones especializadas para cada interfaz de usuario. Mediante la implementación del patrón BFF, aprenderás a personalizar el backend para adaptarse mejor a los requisitos específicos de cada frontend, lo que resulta en una experiencia de usuario más fluida y eficiente.

En el desarrollo de aplicaciones modernas, especialmente en entornos donde múltiples tipos de dispositivos necesitan interactuar con los mismos sistemas, surge el desafío de gestionar adecuadamente las necesidades específicas de cada frontend. El patrón Backend for Frontend (BFF) aparece precisamente como una solución arquitectónica para enfrentar estos retos. Antes de adentrarnos en los detalles de este patrón, es importante entender qué es un patrón de diseño en el contexto del desarrollo de software.

Con esta base en mente, el patrón BFF se presentará como una respuesta específica a las dificultades que surgen cuando un solo backend intenta adaptarse a las diversas necesidades de múltiples interfaces, como aplicaciones de escritorio, móviles y web. A través de este patrón, se puede personalizar y optimizar la comunicación entre el backend y cada tipo de frontend, mejorando tanto el rendimiento como la experiencia del usuario.

Resultado de aprendizaje

El estudiante será capaz de:

RA2. Desarrolla e implementa patrones arquitectónicos conocidos, a partir del diseño, que den respuesta a diferentes frontends.

Indicador de logro:

IL4. Aplica el patrón arquitectónico Backend for Frontend (BFF) para la integración eficiente entre múltiples frontends y backends.

Conceptos relevantes

Backend	BFF	Frontend
Patrón de diseño		

Preguntas activadoras

- ¿Cuáles son las ventajas de implementar un backend personalizado para cada tipo de frontend?
- ¿En qué situaciones crees que el patrón BFF podría no ser la mejor solución?
- ¿Qué diferencias clave existen entre los requisitos de un frontend móvil y una aplicación web de escritorio?
- ¿Qué beneficios aporta el BFF al usuario final en términos de rendimiento y experiencia?

Actividad

Descripción de la actividad

En esta cuarta semana realizarás una actividad formativa grupal, llamada "Aplicando el patrón arquitectónico Backend for Frontend (BFF)", donde deberás implementar el patrón arquitectónico Backend for Frontend (BFF) para optimizar la interacción entre múltiples frontends y backends. Además, tendrás que analizar las ventajas y desventajas de este patrón, señalando en qué escenarios resulta más efectivo.

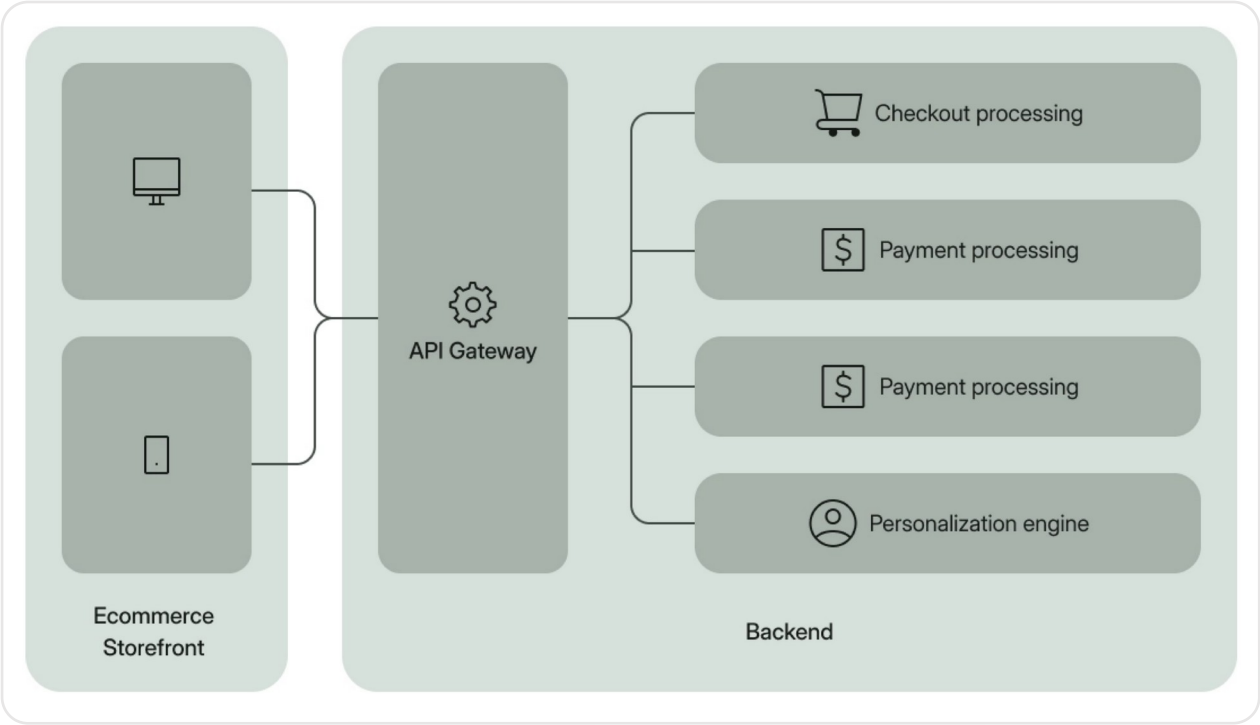
Analizando el patrón Backend for Frontend (BFF)

Introducción

En el desarrollo de aplicaciones modernas, la interacción eficiente entre múltiples frontends (como aplicaciones web, móviles y otros dispositivos) y un backend centralizado plantea desafíos únicos. Cada tipo de frontend tiene requisitos específicos en términos de datos, tiempos de respuesta y formato, lo que puede generar problemas de rendimiento y complejidad en un backend que intente cubrir todas estas necesidades de manera uniforme.

Figura 1

Backend for Frontend (BFF)



Nota. La figura representa el patrón de diseño Backend for Frontend (BFF). Peplinski, J. (2024). *Backend for Frontend (BFF): What You Need to Know.* https://alokai.com/blog/backend-for-frontend

El Patrón arquitectónico Backend for Frontend (BFF) surge como una solución a estos desafíos, pero antes de definirlo, debes entender qué es un Patrón de Diseño.

Un **patrón de diseño** es una solución probada para problemas comunes en el desarrollo de software. Te proporciona una estructura o guía para organizar el código y resolver desafíos de manera eficiente y reutilizable. Utilizar patrones de diseño es útil porque te permite abordar problemas complejos con soluciones ya establecidas y entendidas, facilitando la colaboración y el mantenimiento del código. Además, aplicarlos ayuda a mejorar la calidad del software, haciéndolo más escalable, fácil de entender y flexible frente a cambios futuros.

Figura 2

Introducción a los patrones de diseño



Nota. La imagen describe los patrones de diseño, además de información sobre sus creadores, elementos clave y su clasificación. Xhádev. (2020). *¿Qué es un patrón de diseño?* [Facebook]. https://www.facebook.com/Xhadev

Relación entre el BFF y patrones de diseño:

El patrón arquitectónico BFF puede usar patrones de diseño en su implementación para resolver problemas específicos en la comunicación y organización interna de los componentes. Por ejemplo:

- Adapter: en un BFF, se podría usar este patrón para adaptar los datos obtenidos de un backend generalizado al formato requerido por un frontend específico.
- Proxy: puede implementarse en el BFF para controlar el acceso o cachear respuestas de otros servicios antes de enviarlas al frontend.
- Strategy: puede ser útil cuando un BFF necesita manejar múltiples estrategias para procesar datos según el tipo de frontend.

Teniendo en cuenta lo anterior, es fundamental comprender que todo patrón de diseño nace desde un problema comun que ya tiene una solución propuesta.

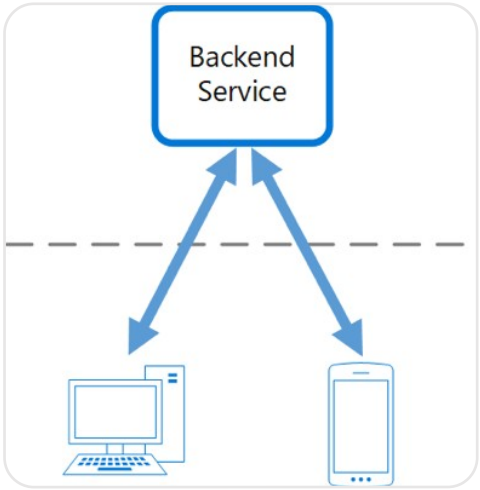
Contexto de origen

Imagina que comienzas desarrollando una aplicación para escritorio con un backend diseñado específicamente para satisfacer sus necesidades. A medida que la base de usuarios crece, decides expandir la experiencia a dispositivos móviles, de modo que el backend debe adaptarse también a esta nueva interfaz. Sin embargo, pronto notarás que las capacidades de los dispositivos móviles son muy diferentes a las de un navegador de escritorio. Tienen pantallas más pequeñas, menos potencia y ciertas limitaciones visuales. Esto significa que el backend ahora debe responder a necesidades diferentes para cada tipo de interfaz, lo que empieza a complicar la estructura y el desarrollo.

Con el tiempo, surgen requisitos que se contraponen entre sí. Lo que es necesario para optimizar la experiencia de escritorio no siempre es ideal para móviles, y viceversa. Esto lleva a cambios constantes en el backend para satisfacer ambos entornos, generando desafíos para el equipo. Con dos equipos trabajando en cada frontend por separado, el backend se convierte en un punto central de coordinación y, a veces, en un cuello de botella. Así, el esfuerzo requerido para mantener un backend funcional para ambos tipos de usuarios puede volverse considerable, afectando el ritmo y la eficiencia del desarrollo.

Figura 3

Arquitectura básica de un servicio backend



Nota. La figura ilustra un servicio backend que interactúa con diferentes dispositivos, a través de una conexión unificada. Microsoft (s.f.). Lean Microsoft. Patrón Backends for Frontends. https://learn.microsoft.com/es-es/azure/architecture/patterns/backends-for-frontends

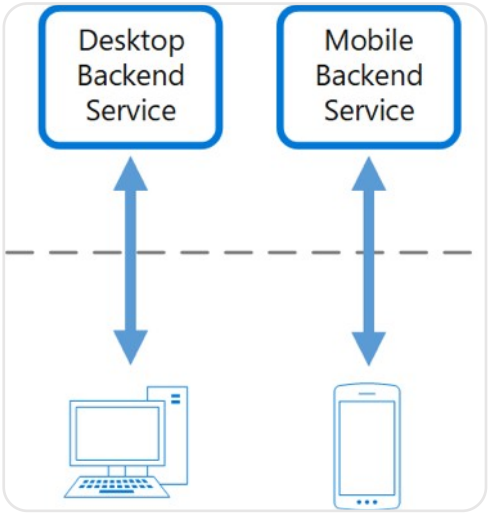
Dado el problema anterior, un pensamiento intuitivo seria: “¿Por qué no crear un backend para cada cliente?”. Con esta idea es que nace el patrón Backend for Frontend (BFF).

¿Qué es el patrón BFF?

El patrón BFF se refiere a una arquitectura en la cual se crea un backend específico para cada tipo de frontend en un sistema (como aplicaciones web, móviles o de IoT). La finalidad de este enfoque es personalizar y optimizar la comunicación y el flujo de datos entre el backend y el frontend específico, proporcionando solo la información y el formato necesarios para cada interfaz de usuario.

Figura 4

Patrón Backend for Frontend



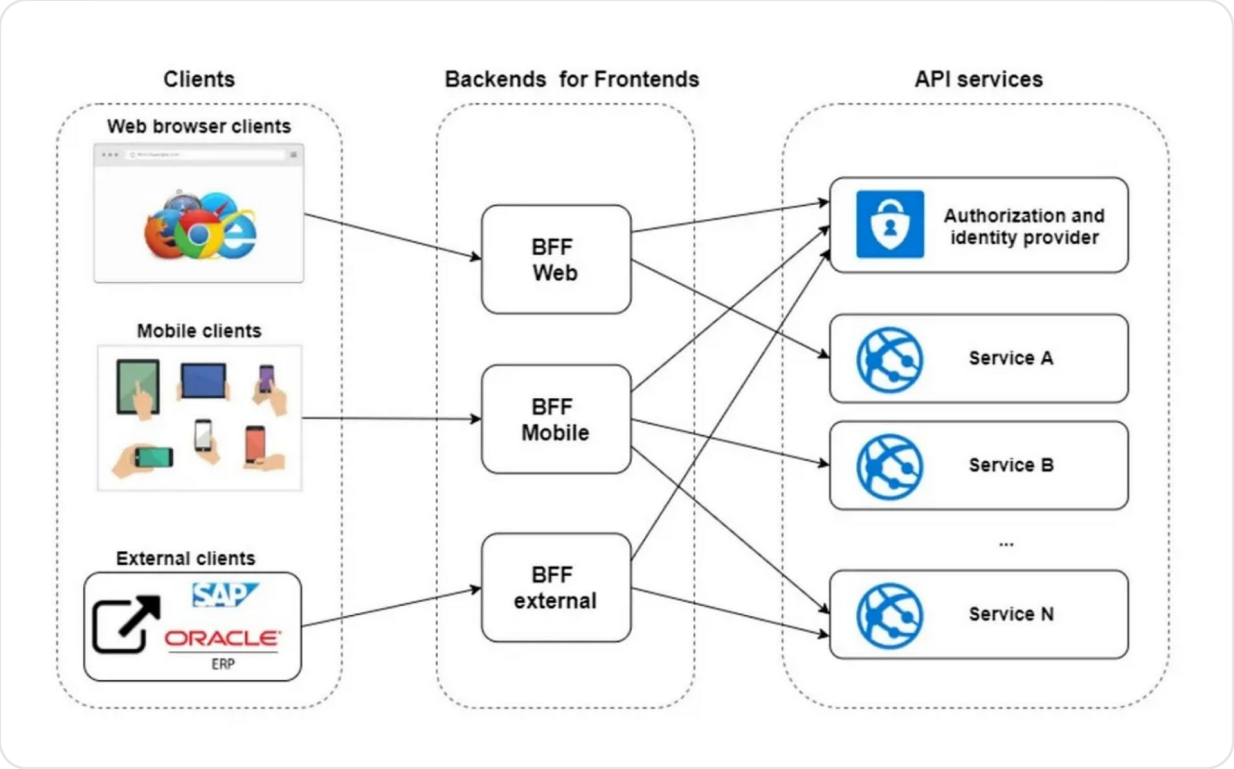
Nota. La imagen muestra la creación de backends específicos para sus clientes (web y mobile). Microsoft (s.f.). *Lean Microsoft. Patrón Backends for Frontends.* https://learn.microsoft.com/es-es/azure/architecture/patterns/backends-for-frontends

¿Para qué se usa?

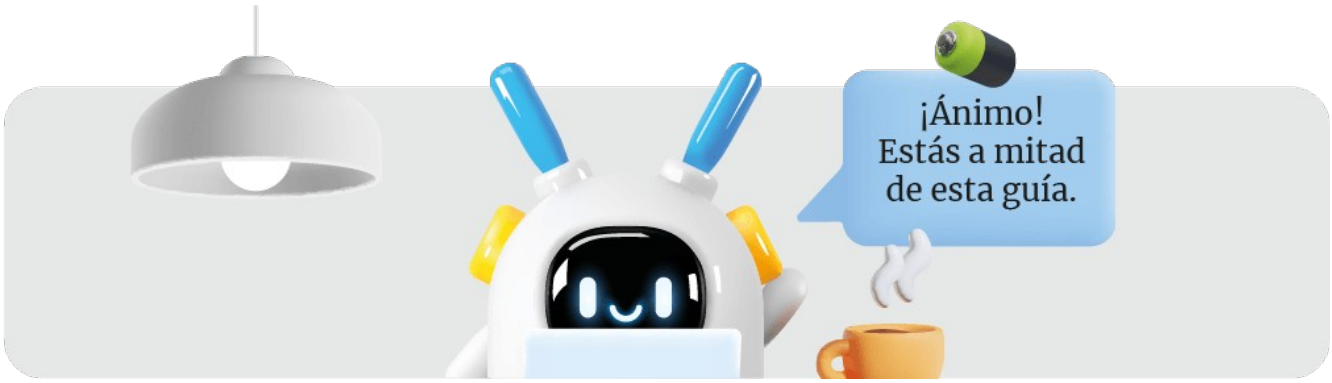
El patrón BFF se usa principalmente para mejorar la experiencia del usuario y el rendimiento de los sistemas que tienen múltiples frontends con requisitos diferentes. Al contar con un backend dedicado para cada frontend, se pueden gestionar los datos y las peticiones de forma más eficiente, evitando la sobrecarga en el frontend y minimizando la lógica de transformación que el frontend tendría que ejecutar.

Figura 5

Esquema de un patrón BFF



Nota. La figura ilustra una arquitectura BFF que separa las interfaces de backend por tipo de cliente (web, móvil y externo) para gestionar sus necesidades específicas. Uzcategui, L. (2023). *Patrón BFF (Backend For Frontend) usando API Gateway.* https://medium.com/@leuzga/patr%C3%B3n-bff-backend-for-frontend-usando-api-gateway-d07098c97465



¿Cómo se utiliza?

Implementarlo, implica que identifiques las necesidades de cada frontend, crees backends personalizados y gestiones la transformación de datos, garantizando también la autenticación adecuada para cada interfaz.

A continuación, se detallan los pasos clave para implementar el BFF de manera efectiva.

- **Identificación de frontends y requisitos específicos**: comienza por identificar cada tipo de frontend en tu sistema y analiza sus necesidades específicas de datos y tiempos de respuesta.
- **Creación de backends específicos para cada frontend**: diseña e implementa un backend separado para cada frontend, en el cual defines los endpoints específicos que satisfacen los requisitos del frontend.
- **Transformación y optimización de datos en el BFF**: cada BFF se encarga de transformar y optimizar los datos antes de enviarlos al frontend, adaptando el formato y la cantidad de datos necesarios.
- **Gestión de seguridad y autenticación**: asegura que cada BFF gestione la autenticación y autorización de forma adecuada, protegiendo los datos según los permisos del usuario y la sensibilidad del frontend.

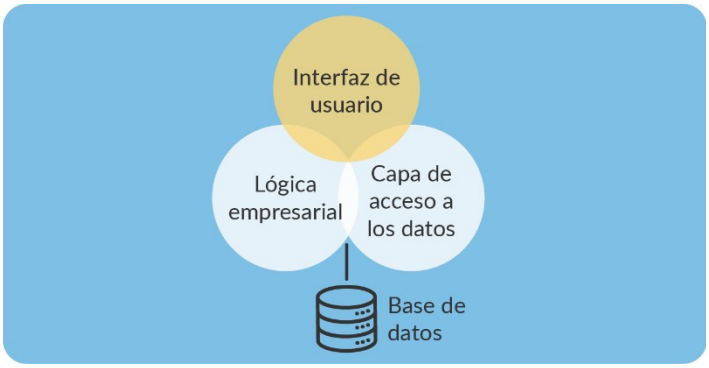
Impacto de no tener un BFF

Sin un BFF, un único backend debe gestionar todas las peticiones de los diferentes frontends, lo que puede llevar a varios problemas:

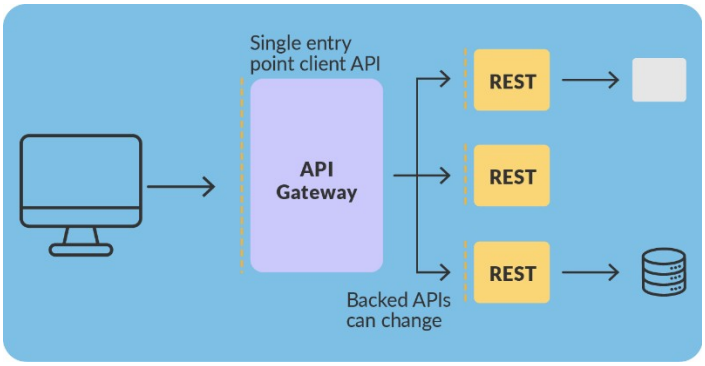
- **Carga de trabajo en el frontend**: los frontends pueden tener que manejar más datos de los necesarios y transformar la información, lo que aumenta la complejidad y reduce la eficiencia.
- **Mala experiencia del usuario**: los tiempos de respuesta pueden no ser óptimos para todos los dispositivos, especialmente en frontends de baja capacidad, como las aplicaciones móviles.

- **Mayor complejidad en el backend**: un solo backend centralizado debe satisfacer los requerimientos de cada frontend, lo que incrementa la complejidad y dificulta el mantenimiento.

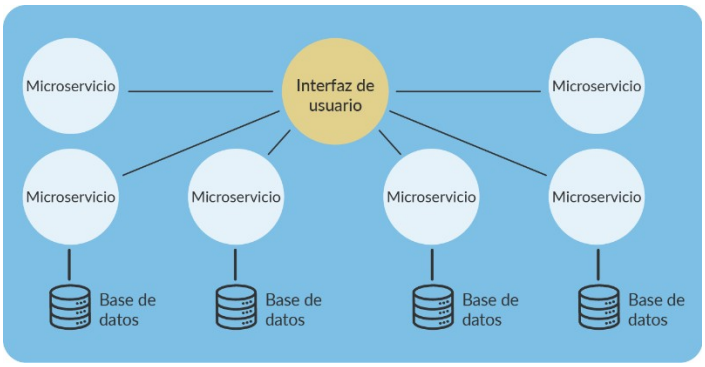
Existen otras arquitecturas que pueden convivir o coexistir en un sistema de software, pero no resuelven exactamente la misma necesidad que busca resolver un BFF. Aquí te mencionamos estas arquitecturas para que comprendas la relación con BFF:



Monolítica
En una arquitectura monolítica, todo el backend es una unidad, lo cual es difícil de optimizar para diferentes frontends. A diferencia del BFF, un monolito no permite personalizar fácilmente la respuesta a cada tipo de interfaz.



API Gateway centralizado
Aunque un API Gateway puede gestionar múltiples frontends, no proporciona la personalización que ofrece el BFF. Un BFF optimiza los datos específicamente para un frontend, mientras que un API Gateway centralizado suele ser una capa de enrutamiento sin lógica personalizada.



Microservicios: una arquitectura de microservicios permite modularizar la aplicación, pero el patrón BFF puede funcionar sobre microservicios, gestionando datos específicos para cada frontend desde su propio backend dedicado, ofreciendo una mayor personalización.

Caso ejemplo de monolito

En el siguiente caso, revisaremos cómo se aplica el patrón BFF; esto te ayudará a comprender por qué no tener un BFF podría generar problemas de mantención en el software.

Links de Interés

En el siguiente enlace encontrarás el proyecto monolítico, correspondiente a una API que te permite listar artículos de un blog. Cada artículo tiene un id, título, contenido e imagen: https://ava.duoc.cl/bbcswebdav/xid-5104851_1

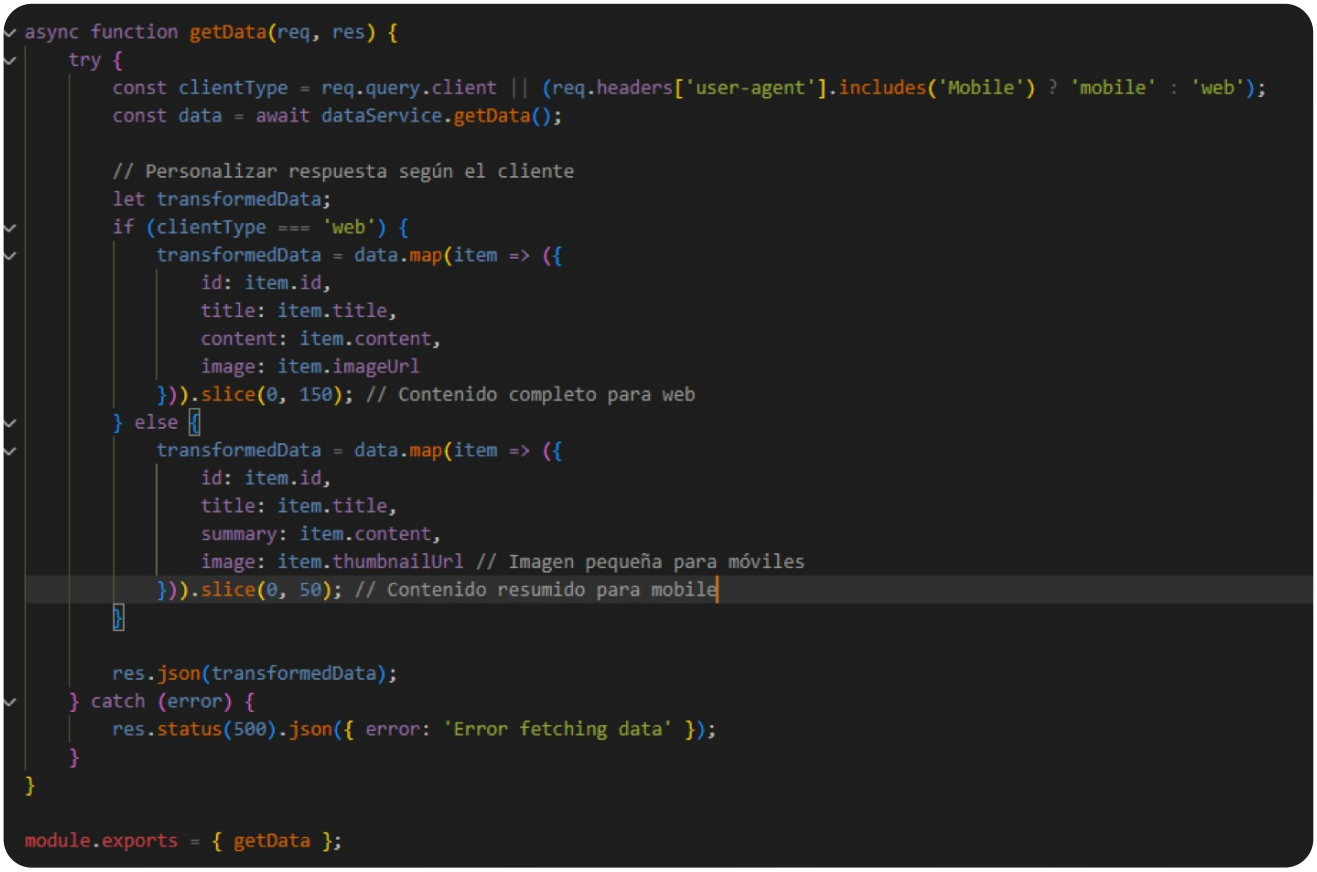
Actualmente, el servicio contiene más de 100 artículos disponibles para los consumidores de la API (web y móvil).

Esto puede generar problemas en dispositivos móviles, ya que la experiencia de navegación podría verse afectada según los niveles de señal disponibles en estos dispositivos.

Para mitigar el problema, el equipo de desarrollo decidió agregar un condicional que, según los encabezados de la petición, se hará una entrega parcial o total de la lista de artículos.

Figura 6

Patrón Backend for Frontend



```
async function getData(req, res) {
    try {
        const clientType = req.query.client || (req.headers['user-agent'].includes('Mobile') ? 'mobile' : 'web');
        const data = await dataService.getData();

        // Personalizar respuesta según el cliente
        let transformedData;
        if (clientType === 'web') {
            transformedData = data.map(item => ({
                id: item.id,
                title: item.title,
                content: item.content,
                image: item.imageUrl
            })).slice(0, 150); // Contenido completo para web
        } else {
            transformedData = data.map(item => ({
                id: item.id,
                title: item.title,
                summary: item.content,
                image: item.thumbnailUrl // Imagen pequeña para móviles
            })).slice(0, 50); // Contenido resumido para mobile
        }

        res.json(transformedData);
    } catch (error) {
        res.status(500).json({ error: 'Error fetching data' });
    }
}

module.exports = { getData };
```

Nota. La imagen muestra la función condicional que permite entregar una cantidad diferente de artículos dependiendo de si el cliente es web o móvil. Microsoft (2006). Power Shell. (7.4.5). [Software]. https://learn.microsoft.com/en-us/powershell/

Estrategias de implementación de BFF

¿Qué estrategias podrías seguir para implementar BFF? A continuación, se listan algunas opciones que dependen en gran medida de los estándares de desarrollo de la empresa, arquitectura de negocios implementada, necesidades de independencia de los equipos, entre otros factores que deben ser analizados a la hora de elegir una estrategia de implementación de BFF.

1. **Crear backends independientes por cada tipo de cliente**

- **Desarrolla un BFF específico para cada cliente** (como web y móvil), y asegúrate de que cada uno tenga su lógica adaptada a las necesidades de su interfaz. Esto puede traducirse en repositorios de código completamente independientes.

- **Aislamiento de lógica de negocio**. mueve cualquier lógica específica del cliente al BFF, evitando duplicarla en el frontend. Esto permitirá que cada BFF adapte la información sin recargar al backend principal.

2. **Diseño de endpoints personalizados**

   - **Endpoints separados**: diseña endpoints específicos para cada tipo de cliente, como /web/articles y /mobile/articles, lo que permite adaptarse mejor a las necesidades de cada frontend.
   - **Evita sobrecargar el backend principal**: cada BFF actúa como un intermediario que optimiza y organiza la información, reduciendo así la dependencia del backend principal.

3. **Aprovechar microservicios y delegar funciones al BFF**

   - **Integración con microservicios**: si el backend usa microservicios, el BFF puede actuar como una capa que combina respuestas de múltiples servicios antes de enviarlas al frontend.
   - **Delegación de funciones**: delegar en el BFF funciones como agregar lógica de presentación o resumir datos permite al backend principal concentrarse en lógica de negocio más general.

Cierre de la semana



Esta semana, exploraste cómo el patrón arquitectónico Backend for Frontend (BFF) optimiza la interacción entre múltiples frontends (como aplicaciones web y móviles) y el backend. El BFF es una solución que permite crear un backend específico para cada tipo de cliente, adaptando la respuesta de manera eficiente a las necesidades de cada interfaz y mejorando la experiencia de usuario. Este enfoque te ayuda a superar las limitaciones de un backend generalizado, donde la personalización para distintos frontends puede convertirse en un desafío, generando problemas de rendimiento y mantenibilidad.

La guía de esta semana comparó el BFF con otras arquitecturas comunes, como los API Gateways y microservicios. Mientras que un API Gateway centralizado facilita el enrutamiento sin personalización de datos, el BFF permite adaptar cada backend a los requerimientos específicos de cada frontend. Esto ofrece una experiencia personalizada para cada usuario y mejora el rendimiento, especialmente en dispositivos con limitaciones, como los móviles. Además, el BFF puede integrarse fácilmente con una arquitectura de microservicios, permitiendo aprovechar la modularidad mientras se optimiza la respuesta por cliente.

Referencias

- Microsoft (s.f.). Lean Microsoft. Patrón Backends for Frontends. https://learn.microsoft.com/es-es/azure/architecture/patterns/backends-for-frontends. https://learn.microsoft.com/en-us/azure/architecture/patterns/_images/backend-for-frontend.png

Lecturas de la semana

- Swagger. (s.f.). What is OpenAPI? https://swagger.io/docs/specification/v3_0/about/
- Swagger. (s.f.). Basic Structure. https://swagger.io/docs/specification/v3_0/basic-structure/ BFF con GraphQl. https://graphql-api-gateway.com/graphql-api-gateway-patterns/bff-derived-client-generation

Apuntes



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.